

AWS SAA-C03

Serverless & Application Integration

SQS, SNS, EventBridge, Kinesis, API Gateway, Step Functions

Solutions Architect Associate

Dispensa di Studio Approfondita

Amazon SQS: code di messaggi

Amazon SQS (Simple Queue Service) e' un servizio di message queuing completamente gestito che permette di disaccoppiare i componenti di un'applicazione. Un produttore invia messaggi alla coda, un consumatore li legge e li elabora. Se il consumatore e' temporaneamente non disponibile, i messaggi restano in coda fino a quando non viene ripristinato.

Standard Queue vs FIFO Queue

Le Standard Queue offrono throughput praticamente illimitato e sono la scelta predefinita. Hanno due caratteristiche importanti da ricordare: at-least-once delivery (un messaggio potrebbe essere consegnato piu' di una volta, quindi il consumatore deve essere idempotente) e best-effort ordering (l'ordine dei messaggi non e' garantito).

Le FIFO Queue garantiscono che i messaggi siano consegnati esattamente una volta (exactly-once processing) e nell'ordine esatto in cui sono stati inviati. Il throughput e' limitato a 300 messaggi al secondo senza batching, o 3000 con batching. Il nome della coda deve terminare con .fifo.

Le FIFO Queue usano due concetti importanti: il Message Group ID raggruppa i messaggi che devono mantenere l'ordine tra loro (messaggi con group ID diversi possono essere elaborati in parallelo), e il Deduplication ID previene l'invio di messaggi duplicati in una finestra di 5 minuti.

SUGGERIMENTO ESAME: "Ordine garantito dei messaggi" = FIFO Queue. "Throughput massimo" = Standard Queue. "Elaborazione esattamente una volta" = FIFO Queue.

Visibility Timeout e Dead Letter Queue

Quando un consumatore legge un messaggio dalla coda, il messaggio non viene cancellato immediatamente ma diventa invisibile per un periodo chiamato Visibility Timeout (default 30 secondi). Se il consumatore elabora il messaggio con successo, lo cancella esplicitamente. Se il consumatore fallisce o va in timeout, il messaggio torna visibile e puo' essere elaborato da un altro consumatore.

Se un messaggio fallisce ripetutamente (il numero di tentativi e' configurabile con maxReceiveCount), viene spostato in una Dead Letter Queue (DLQ). La DLQ e' una coda separata dove puoi analizzare i messaggi problematici, fare debug e decidere come gestirli. E' una best practice configurare sempre una DLQ.

Il Long Polling riduce il numero di chiamate API vuote. Invece di ritornare immediatamente se la coda e' vuota, SQS aspetta fino a 20 secondi per un messaggio. Questo riduce i costi e migliora l'efficienza.

SUGGERIMENTO ESAME: "Messaggi che falliscono ripetutamente" = Dead Letter Queue. "Ridurre le"

chiamate API a SQS" = Long Polling.

SQS con Lambda e Auto Scaling

Lambda può essere configurato come consumatore di una coda SQS. Lambda poll automaticamente la coda e invoca la funzione per ogni batch di messaggi. Se l'elaborazione fallisce, i messaggi tornano visibili nella coda.

Per scalare consumatori EC2 in base alla lunghezza della coda, usa la metrica CloudWatch `ApproximateNumberOfMessagesVisible` come trigger per un Auto Scaling Group. Più messaggi in coda, più istanze vengono lanciate.

Amazon SNS: pub/sub messaging

Amazon SNS (Simple Notification Service) implementa il pattern publish/subscribe. Un publisher invia un messaggio a un Topic, e SNS lo distribuisce a tutti i subscriber del topic. A differenza di SQS dove un messaggio viene elaborato da un solo consumatore, con SNS lo stesso messaggio raggiunge tutti i subscriber.

I subscriber possono essere: code SQS, funzioni Lambda, endpoint HTTP/HTTPS, email, SMS, e notifiche push mobile. Questo permette il pattern fan-out: un singolo evento puo' attivare elaborazioni multiple in parallelo.

Il pattern SNS + SQS fan-out e' molto comune: un messaggio viene pubblicato su un topic SNS che ha multiple code SQS come subscriber. Ogni coda riceve una copia del messaggio e puo' elaborarlo indipendentemente. Questo disaccoppia completamente il publisher dai consumatori.

SNS supporta anche FIFO Topics (solo con SQS FIFO come subscriber) e Message Filtering: ogni subscriber puo' definire una filter policy per ricevere solo i messaggi che corrispondono a certi attributi.

**SUGGERIMENTO ESAME: "Un evento deve attivare piu' elaborazioni in parallelo" = SNS fan-out.
"Notificare piu' sistemi di un evento" = SNS Topic.**

Amazon EventBridge: event bus serverless

EventBridge (precedentemente CloudWatch Events) e' un event bus serverless che connette applicazioni usando eventi. E' l'evoluzione di SNS per architetture event-driven complesse.

EventBridge ha tre tipi di event bus: il Default Event Bus riceve eventi dai servizi AWS (un'istanza EC2 che cambia stato, un oggetto caricato su S3, ecc.), i Custom Event Bus ricevono eventi dalle tue applicazioni, e i Partner Event Bus ricevono eventi da applicazioni SaaS di terze parti (Zendesk, Datadog, Auth0, ecc.).

Le Rules definiscono quali eventi catturare (tramite pattern matching sul contenuto dell'evento) e dove inviarli (target). I target possono essere Lambda, SQS, SNS, Step Functions, Kinesis, e molti altri servizi AWS.

EventBridge ha funzionalita' avanzate che SNS non ha: Schema Registry (scopre e registra automaticamente la struttura degli eventi), Archive and Replay (archivia gli eventi e permette di riprodurli per debug o recovery), e Scheduler (cron e rate expressions per eventi schedulati).

SUGGERIMENTO ESAME: "Reagire a eventi dei servizi AWS" = EventBridge. "Integrare eventi da applicazioni SaaS" = EventBridge Partner Event Bus. "Schedulare task periodici serverless" = EventBridge Scheduler.

Amazon Kinesis: streaming di dati in tempo reale

Kinesis e' una famiglia di servizi per raccogliere, elaborare e analizzare dati in streaming in tempo reale.

Kinesis Data Streams

Kinesis Data Streams e' il servizio core per l'ingestion di dati in streaming. I dati sono organizzati in Shards: ogni shard supporta 1 MB/sec in ingresso e 2 MB/sec in uscita. Aggiungi shard per aumentare la capacita'.

I dati restano nello stream per un periodo di retention (default 24 ore, massimo 365 giorni). Piu' consumatori possono leggere gli stessi dati indipendentemente. I consumatori possono essere applicazioni custom con Kinesis Client Library (KCL), funzioni Lambda, o Kinesis Data Analytics.

Enhanced Fan-Out permette a ogni consumatore di avere 2 MB/sec dedicati per shard (invece di condividere i 2 MB/sec tra tutti i consumatori). Usa un modello push invece di pull.

SUGGERIMENTO ESAME: "Streaming di dati in tempo reale con elaborazione custom" = Kinesis Data Streams. "Piu' consumatori che leggono gli stessi dati" = Kinesis Data Streams.

Kinesis Data Firehose

Firehose e' il modo piu' semplice per caricare dati in streaming verso destinazioni come S3, Redshift, OpenSearch e Splunk. E' completamente gestito: non devi gestire shard o capacita'.

Firehose puo' trasformare i dati in transito usando Lambda (es. convertire formato, arricchire dati) e puo' convertire il formato (es. JSON a Parquet). Il delivery e' near real-time con un buffer minimo di 60 secondi.

SUGGERIMENTO ESAME: "Caricare dati in streaming su S3 automaticamente" = Kinesis Data Firehose. "Trasformare dati in streaming prima di salvarli" = Firehose con Lambda transformation.

Kinesis Data Analytics

Kinesis Data Analytics permette di eseguire query SQL o applicazioni Apache Flink su dati in streaming. Puoi calcolare aggregazioni, rilevare anomalie e generare metriche in tempo reale.

SUGGERIMENTO ESAME: "Analisi SQL su dati in streaming" = Kinesis Data Analytics.

Amazon API Gateway

API Gateway e' un servizio completamente gestito per creare, pubblicare e gestire API. Funziona come "front door" per le tue applicazioni, gestendo autenticazione, throttling, caching e monitoring.

Tipi di API

REST API e' il tipo piu' completo. Supporta caching delle risposte (per ridurre le chiamate al backend), throttling (10000 richieste/secondo di default, configurabile), API keys e usage plans (per limitare e monetizzare l'accesso), integrazione con WAF, e trasformazione di richieste/risposte con mapping templates.

HTTP API e' piu' semplice e costa circa il 70% in meno. E' ottimizzato per proxy verso Lambda e backend HTTP. Non supporta caching o trasformazioni complesse, ma e' sufficiente per molti casi d'uso.

WebSocket API supporta comunicazione bidirezionale in tempo reale. Il client mantiene una connessione persistente con API Gateway, che puo' inviare messaggi al client in qualsiasi momento. Ideale per chat, gaming, dashboard live.

Autenticazione e autorizzazione

API Gateway supporta diversi metodi di autenticazione: IAM (per chiamate da altri servizi AWS o SDK), Lambda Authorizer (una funzione Lambda che valida un token custom e restituisce una policy IAM), e Cognito User Pools (valida JWT token emessi da Cognito).

Deployment e stages

Le API vengono deployate in Stages (es. dev, staging, prod). Ogni stage ha il suo URL e puo' avere configurazioni diverse (throttling, caching, variabili di stage). Canary deployment permette di instradare una percentuale del traffico verso una nuova versione per testarla gradualmente.

SUGGERIMENTO ESAME: "API REST con caching e throttling" = API Gateway REST API. "Proxy semplice verso Lambda con costo minimo" = HTTP API. "Comunicazione real-time bidirezionale" = WebSocket API.

AWS Step Functions: orchestrazione di workflow

Step Functions permette di coordinare componenti distribuiti in workflow visuali. Definisci una state machine con stati che possono essere: Task (invoca Lambda, ECS, API, ecc.), Choice (branching condizionale), Parallel (esecuzione parallela), Wait (pausa), Map (iterazione su array), e altri.

Standard vs Express Workflows

Standard Workflows possono durare fino a un anno, garantiscono exactly-once execution, e mantengono uno storico completo di ogni esecuzione. Sono ideali per workflow di lunga durata come approvazioni, orchestrazione di microservizi, e processi ETL.

Express Workflows durano massimo 5 minuti, hanno at-least-once execution, e sono ottimizzati per alto volume (oltre 100000 esecuzioni al secondo). Costano molto meno degli Standard. Ideali per elaborazione di eventi IoT, trasformazioni di dati ad alto throughput.

SUGGERIMENTO ESAME: "Orchestrare piu' Lambda in sequenza con gestione degli errori" = Step Functions. "Workflow di approvazione con attesa di input umano" = Step Functions Standard. "Elaborazione ad alto volume di breve durata" = Step Functions Express.

AWS AppSync: GraphQL gestito

AppSync è un servizio gestito per creare API GraphQL. A differenza di REST dove ogni endpoint restituisce una struttura fissa, GraphQL permette al client di specificare esattamente quali dati vuole. AppSync si integra nativamente con DynamoDB, Lambda, RDS, OpenSearch e HTTP endpoints.

AppSync supporta subscriptions per aggiornamenti in tempo reale: il client si sottoscrive a certi eventi e riceve notifiche push quando i dati cambiano. Supporta anche offline sync per applicazioni mobile.

SUGGERIMENTO ESAME: "API GraphQL gestita" = AppSync. "Sincronizzazione dati offline per app mobile" = AppSync con Amplify.